Vamos a mejorar nuestro registro de alumnos

Antes de modificar

Nombre: _____ Grupo: _____

Propósito. Localizar las partes de una aplicación en Python y realizar cambios pequeños, entendiendo cómo se conectan el código, los formularios y la base de datos.

Trabajarás sobre la carpeta CRUD_Alumnos de la práctica anterior. Al terminar, tu aplicación tendrá un encabezado personalizado, una página de ayuda con la fecha y un nuevo dato: el turno del alumno.

1. Prepara tu carpeta

Detén la aplicación con Ctrl + C en la terminal. Cierra la carpeta en Visual Studio Code y, en el explorador de archivos, copia CRUD_Alumnos. Nombra la copia CRUD_Alumnos_Practica02. Abre esa copia desde Archivo → Abrir carpeta. Conserva la original como respaldo del código.

La copia de la carpeta seguirá conectada a la misma base escuela_practica. No es un respaldo de los registros. Para las pruebas usa nombres ficticios y elimina solamente el registro de prueba que tú agregues.

2. Comprueba que funciona antes de empezar

En Windows, abre Terminal → Nueva terminal. Verifica que estás dentro de la copia y ejecuta:

```
.\.venv\Scripts\python.exe app.py
```

Si la copia no tiene la carpeta .venv, prepara el entorno una sola vez:

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe app.py
```

Abre http://127.0.0.1:5000 en el navegador. MongoDB debe estar iniciado. No abras index.html con doble clic ni uses Live Server: las páginas deben abrirse a través de Flask.

Primero localiza; después modifica

Reconocimiento del proyecto

En el panel izquierdo de Visual Studio Code, abre cada archivo. No cambies nada todavía. Usa Ctrl + F para buscar dentro del archivo abierto; Ctrl + Shift + F busca en toda la carpeta. En Mac, usa Cmd en lugar de Ctrl para estas búsquedas.

Archivo o carpeta	¿Qué encuentras ahí?
app.py	Las herramientas importadas, la conexión con MongoDB y las rutas de la aplicación.
requirements.txt	Los paquetes externos que se instalan: Flask y pymongo.
templates/base.html	El encabezado y la estructura compartida por las páginas.
templates/index.html	El formulario para agregar y la tabla de alumnos.
templates/editar.html	El formulario para cambiar un alumno.
templates/error.html	Los mensajes de error que prepara la aplicación.
static/estilos.css	Los colores, tamaños y distribución de la pantalla.

3. Identifica las herramientas que ya usamos

Abre app.py y observa sus primeras cuatro líneas:

```
from flask import Flask, render_template, request, redirect, url_for, abort
from pymongo import MongoClient
from pymongo.errors import PyMongoError
from bson import ObjectId
```

from … import … indica de dónde traemos una herramienta. Flask organiza la aplicación; render_template carga las páginas; request recibe datos; redirect y url_for ayudan a regresar o ir a otra ruta; abort detiene una petición con un error.

MongoClient conecta con MongoDB. PyMongoError permite atender errores de la base. ObjectId trabaja con la clave de cada registro. bson viene con PyMongo: no instales un paquete separado llamado bson.

En tu libreta: anota qué archivo cambiarías para modificar un color, recibir un dato y agregar una página.

Una dirección, una tarea

Reconocimiento de rutas

En app.py, busca @app.route. Recorre sus coincidencias. Una ruta le indica a Flask qué función ejecutar cuando se solicita una dirección.

```
@app.route("/")
def inicio():
    lista = list(alumnos.find().sort("nombre", 1))
    return render_template("index.html", alumnos=lista)
```

La primera línea señala la dirección. def inicio(): nombra la función. Las líneas con cuatro espacios debajo pertenecen a esa función. La última línea carga index.html y le entrega la lista de alumnos.

Ruta actual	Función	Qué hace
/	inicio	Muestra el formulario y los registros.
/agregar	agregar	Recibe un formulario por POST y guarda un alumno.
/editar/<id>	editar	GET muestra el formulario; POST guarda los cambios.
/eliminar/<id>	eliminar	Recibe una petición POST y elimina ese registro.

No confundas estas tres cosas

Ruta web: /editar/<id> es una dirección de la aplicación. <id> se sustituye por la clave real del alumno.
Ruta de archivo: templates/editar.html indica dónde está guardado el archivo.
Función: editar es el nombre que usamos con url_for para construir el enlace.

GET pide ver una página. POST envía datos, como cuando presionas Guardar alumno. Por eso no se prueba /agregar escribiéndolo directamente en la barra del navegador: eso enviaría GET.

4. Antes del primer cambio

Identifica y anota la ruta, la función y el archivo HTML de la pantalla de edición. Después localiza if __name__ == "__main__": al final de app.py: las nuevas rutas irán antes de ese bloque.

Personaliza la pantalla principal

Cambio 1 · Encabezado

5. Abre templates/base.html. Busca <h1>: ahí está el título visible. Busca <title>: ese texto aparece en la pestaña del navegador. El archivo original está escrito en pocas líneas; lo ordenaremos para localizar mejor sus partes.

6. Reemplaza todo el contenido de base.html por este código. Puedes cambiar “Registro de mi grupo” por “Registro del grupo 501” o por el grupo que corresponda.

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Registro de mi grupo</title>
  <link rel="stylesheet"
        href="{{ url_for('static', filename='estilos.css') }}">
</head>
<body>
  <main>
    <header>
      <p class="etiqueta">PRÁCTICA 02 · APLICACIONES WEB</p>
      <h1>Registro de mi grupo</h1>
      <p>Organizamos los datos de nuestros alumnos.</p>
    </header>
    {% block contenido %}{% endblock %}
    <footer>Laboratorio de Aplicaciones Web · Datos de práctica</footer>
  </main>
</body>
</html>
```

7. Guarda con Ctrl + S y actualiza el navegador. Deben cambiar el título grande y el texto de la pestaña. Si no cambia, confirma que editaste la copia que está ejecutándose.

Observa: block contenido reserva el espacio donde aparecen el formulario o las otras páginas. No borres ese bloque al personalizar el encabezado.

Prepara la página de ayuda

Cambio 2 · Nueva librería y ruta

8. Abre app.py. Busca from bson import ObjectId. En una línea nueva, justo debajo y sin espacios al inicio, agrega:

```
from datetime import datetime
```

datetime nos permite trabajar con fecha y hora. Viene incluido en Python: no se instala con pip ni se agrega a requirements.txt. Esta línea trae la herramienta, pero todavía necesitamos usarla.

9. Busca if __name__ == "__main__":. Justo antes de ese bloque, pega la siguiente ruta. Deja una línea vacía antes y después. No la pegues dentro de otra función.

```
@app.route("/ayuda")
def ayuda():
    fecha = datetime.now().strftime("%d/%m/%Y")
    return render_template("ayuda.html", fecha=fecha)
```

Parte	Qué significa aquí
@app.route("/ayuda")	La nueva página se abrirá en la dirección /ayuda.
def ayuda():	Nombra la función que responde a esa dirección.
datetime.now()	Obtiene la fecha y hora del equipo que ejecuta Python.
strftime("%d/%m/%Y")	Escribe la fecha como día/mes/año.
fecha=fecha	Envía el valor a la página con el nombre fecha.

Revisa la posición

La línea @app.route y la línea def comienzan desde el margen izquierdo. Las dos líneas dentro de ayuda llevan cuatro espacios. El bloque que inicia con if __name__ conserva su posición al final del archivo.

Guarda app.py. Todavía falta crear ayuda.html. Si entras ahora a /ayuda, Flask no encontrará esa plantilla. Completa la siguiente página antes de probar.

Conecta la ayuda con la pantalla

Cambio 2 · Plantilla y enlace

10. En el panel izquierdo, haz clic derecho sobre templates → Nuevo archivo. Escribe ayuda.html. Debe quedar junto a index.html; no dentro de static. Escribe lo siguiente:

```
{% extends 'base.html' %}
{% block contenido %}
<section>
  <h2>Ayuda del registro</h2>
  <p>Fecha del equipo: {{ fecha }}</p>
  <ol>
    <li>Escribe el nombre y el grupo del alumno.</li>
    <li>Presiona Guardar alumno para registrarlo.</li>
    <li>Usa Editar para corregir sus datos.</li>
    <li>Usa Eliminar solo con registros de prueba.</li>
  </ol>
  <a class="boton secundario" href="{{ url_for('inicio') }}">
    Volver al registro
  </a>
</section>
{% endblock %}
```

11. Guarda el archivo. Abre base.html, busca </header> y agrega este bloque justo antes de esa etiqueta, sin borrar lo que ya existe:

```
      <p>
        <a href="{{ url_for('inicio') }}">Inicio</a> |
        <a href="{{ url_for('ayuda') }}">Ayuda</a>
      </p>
```

12. Guarda y abre http://127.0.0.1:5000/ayuda. Debes ver las instrucciones y la fecha del equipo. Haz clic en Inicio y después en Ayuda para comprobar ambos enlaces.

¿Qué conectaste? url_for('ayuda') busca la función ayuda de app.py. Esa función prepara la fecha y carga ayuda.html. La expresión {{ fecha }} muestra el dato recibido. extends reutiliza el encabezado de base.html.

Si aparece TemplateNotFound: revisa que se llame ayuda.html, sin .txt al final, y esté dentro de templates.

Agrega el turno en Python

Cambio 3 · Recibir un nuevo dato

Para guardar un dato nuevo hay que conectarlo en varios lugares: el formulario que lo pide, Python que lo recibe, la tabla que lo muestra y la pantalla que lo permite editar. Trabajaremos en ese orden lógico, empezando por Python.

13. Abre app.py y busca def leer_formulario():. Selecciona desde esa línea hasta su return, sin incluir def buscar_alumno(id):. Reemplaza únicamente esa función por:

```
def leer_formulario():
    nombre = request.form.get("nombre", "").strip()
    grupo = request.form.get("grupo", "").strip()
    turno = request.form.get("turno", "").strip()
    if not nombre or not grupo or not turno:
        abort(400, description="Completa nombre, grupo y turno.")
    if len(nombre) > 80 or len(grupo) > 20:
        abort(400, description="Usa hasta 80 caracteres en nombre y 20 en grupo.")
    if turno not in ["Matutino", "Vespertino"]:
        abort(400, description="Selecciona un turno válido.")
    return {"nombre": nombre, "grupo": grupo, "turno": turno}
```

¿Qué cambió?

request.form.get("turno", "") busca el dato enviado con el nombre turno. Si no llegó, usa un texto vacío. strip() quita espacios sobrantes al principio y al final.

La primera condición revisa que se llenen los tres datos. La nueva condición permite solamente Matutino o Vespertino. El return entrega los tres valores juntos para que puedan guardarse.

14. Busca datos = leer_formulario(). Aparece en agregar y editar. Las dos funciones ya usan lo que devuelve leer_formulario; por eso no necesitas cambiar insert_one ni update_one para incluir turno.

Guarda, pero todavía no pruebes guardar un alumno. Los formularios actuales aún no envían turno. Completa las dos páginas siguientes; de lo contrario aparecerá el aviso de datos incompletos.

En tu libreta: explica por qué no basta con dibujar una caja nueva en HTML para que ese dato se guarde en MongoDB.

Pide el turno y muéstralo en la lista

Cambio 3 · Formulario y tabla

15. Abre templates/index.html. Reemplaza todo su contenido por este código y guarda. Se conserva el registro, la edición y la eliminación.

```
{% extends 'base.html' %}
{% block contenido %}
<section>
  <h2>Agregar alumno</h2>
  <form method="post" action="{{ url_for('agregar') }}" class="registro">
    <label>Nombre completo
      <input name="nombre" required maxlength="80">
    </label>
    <label>Grupo
      <input name="grupo" required maxlength="20">
    </label>
    <label>Turno
      <select name="turno" required>
        <option value="">Selecciona un turno</option>
        <option value="Matutino">Matutino</option>
        <option value="Vespertino">Vespertino</option>
      </select>
    </label>
    <button>Guardar alumno</button>
  </form>
</section>
<section>
  <h2>Alumnos registrados <span class="contador">{{ alumnos|length }}</span></h2>
  <div class="tabla">
    <table>
      <thead><tr>
        <th>Nombre</th><th>Grupo</th><th>Turno</th><th>Acciones</th>
      </tr></thead>
      <tbody>
      {% for alumno in alumnos %}
        <tr>
          <td>{{ alumno.nombre }}</td>
          <td>{{ alumno.grupo }}</td>
          <td>{{ alumno.get('turno', 'Sin asignar') }}</td>
          <td><div class="acciones">
            <a class="boton secundario"
               href="{{ url_for('editar', id=alumno._id) }}">Editar</a>
            <form method="post"
                  action="{{ url_for('eliminar', id=alumno._id) }}"
                  onsubmit="return confirm('¿Eliminar este alumno?');">
              <button class="peligro">Eliminar</button>
            </form>
          </div></td>
        </tr>
      {% else %}
        <tr><td colspan="4">Todavía no hay alumnos registrados.</td></tr>
      {% endfor %}
      </tbody>
    </table>
  </div>
</section>
{% endblock %}
```

Permite cambiar el turno

Cambio 3 · Pantalla de edición

16. Abre templates/editar.html. Reemplaza todo su contenido por este código y guarda:

```
{% extends 'base.html' %}
{% block contenido %}
<section>
  <h2>Editar alumno</h2>
  <p>Corrige los datos y guarda los cambios.</p>
  <form method="post"
        action="{{ url_for('editar', id=alumno._id) }}" class="registro">
    <label>Nombre completo
      <input name="nombre" value="{{ alumno.nombre }}"
             required maxlength="80">
    </label>
    <label>Grupo
      <input name="grupo" value="{{ alumno.grupo }}"
             required maxlength="20">
    </label>
    <label>Turno
      <select name="turno" required>
        <option value="">Selecciona un turno</option>
        {% for opcion in ['Matutino', 'Vespertino'] %}
          <option value="{{ opcion }}"
            {% if alumno.get('turno') == opcion %}selected{% endif %}>
            {{ opcion }}
          </option>
        {% endfor %}
      </select>
    </label>
    <button>Guardar cambios</button>
    <a class="boton secundario" href="{{ url_for('inicio') }}">Cancelar</a>
  </form>
</section>
{% endblock %}
```

17. Regresa a index.html y localiza name="turno". Ese nombre coincide con el que Python recibe. Busca también colspan="4": ahora el mensaje de tabla vacía debe abarcar cuatro columnas.

18. En editar.html, selected deja marcado el turno que ya tiene el alumno. Los registros anteriores a esta modificación mostrarán Sin asignar en la lista; al editarlos, selecciona un turno y guarda.

19. Abre ayuda.html y cambia el primer elemento de la lista por:
Escribe el nombre, el grupo y selecciona el turno.

Prueba cada cambio

Comprobación y entrega

20. Guarda todos los archivos. En la terminal detén con Ctrl + C y vuelve a ejecutar el comando de la página 1. Abre http://127.0.0.1:5000. Realiza estas pruebas en orden y anota el resultado.

Prueba	Resultado que debes observar
1. Abrir Inicio y Ayuda	El encabezado aparece en ambas páginas. Ayuda muestra la fecha del equipo y permite volver.
2. Agregar Ana Prueba / 501 / Matutino	Aparece una fila con los tres datos.
3. Editar a Ana Prueba	El turno Matutino está seleccionado. Cámbialo a Vespertino y guarda: la tabla debe mostrar el cambio.
4. Actualizar el navegador	Los datos siguen ahí; no se pierden al recargar.
5. Guardar sin seleccionar turno	El navegador pide seleccionar una opción. Python también valida el dato si recibe un envío incompleto.
6. Revisar un registro anterior, si existe	Muestra Sin asignar. Al editarlo, puedes elegir un turno.
7. Eliminar solo a Ana Prueba	Pide confirmación y, al aceptar, desaparece de la tabla.

Corrige y explica lo que hiciste

Revisión final

Si algo no funciona

Mensaje o situación	Qué revisar
IndentationError / SyntaxError	Espacios, comillas, paréntesis y dos puntos. Compara el bloque completo con la guía.
NameError: datetime...	La nueva importación debe estar arriba de app.py y escrita exactamente como en la guía.
BuildError / página 404	url_for usa el nombre de la función: ayuda. La dirección del navegador termina en /ayuda.
No fue posible trabajar con MongoDB	El servidor MongoDB debe estar iniciado. Tener Compass abierto no sustituye al servidor.
No aparece turno o no se guarda	Guarda ambos HTML y app.py. Comprueba name="turno" y la clave "turno" del return.

Entrega

Presenta tres capturas: registro con turno, edición con turno seleccionado y página de ayuda. Adjunta app.py, base.html, index.html, editar.html y ayuda.html conservando la carpeta templates. No incluyas .venv.

Responde en tu libreta: 1) ¿Dónde se importan las herramientas? 2) ¿Qué conecta /ayuda con ayuda.html? 3) ¿Por qué el nombre turno debe coincidir en HTML y Python?